TP1 : cadrer une application IA et préparer l'environnement

La démarche pas à pas, étape par étape et test par test

A. Larhlimi

Développement et déploiement d'applications intelligentes

Semaine 1 · TP de 2 h · en binôme

Objectif du TP

Cadrer avant de coder

La première question n'est pas « quel modèle ? » mais « faut-il vraiment un modèle ? ».

Trois cas à trancher avec une grille à quatre critères, et à justifier par écrit.

Un dépôt qui survit à votre portable

Un modèle dans un notebook n'est pas un produit : personne ne peut l'exécuter ni le tester.

À la fin : un dépôt que l'enseignant clone et fait tourner en trois commandes.

Ce que vous installez aujourd'hui sert les dix semaines suivantes.

L'application à construire : le fil rouge

Une équipe relation client reçoit des milliers d'avis par jour et les traite dans l'ordre d'arrivée. On veut faire remonter les **avis négatifs** en tête de file.

Semaine	Ce qui s'ajoute	Semaine	Ce qui s'ajoute
1	cadrage, environnement, dépôt, tests	6 à 8	paquet, Docker, CI/CD
2	le modèle (TF-IDF, régression logistique)	9	suivi de la dérive, réentraînement
3	une API REST	10	agents : le modèle devient un outil
5	une interface de démonstration	11	projet et soutenance

Problème volontairement simple côté modèle : la difficulté est dans le déploiement et le suivi.

Les huit étapes, les huit contrôles

	Étape	Qui	Contrôle
1	Décompresser le kit	chacun	T0 : les 7 dossiers, `.gitignore` présent
2	Créer l'environnement virtuel	chacun	T1 : 3 failed, 31 passed, 4 skipped
3	Remplir les trois fiches	le binôme	T2 : 12 passed
4	Créer le dépôt, pousser le kit	membre A	T3 : aucune ligne `.venv/`
5	Installer l'environnement complet	chacun	T4 : 0 erreur(s) au diagnostic
6	Un commit par membre	chacun	T5 : deux auteurs, 3 passed
7	Vérification finale	le binôme	T6 : 38 passed, CI verte
8	Le test de l'enseignant	le binôme	T7 : 38 passed sur un clone frais

Ne passez à l'étape suivante que lorsque le contrôle précédent est vert.

Avant la séance : quatre réglages

Python 3.11, Git, un compte GitHub. Puis, une fois pour toutes :

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@ecole.ma"
git config --global init.defaultBranch main
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Répartition des rôles

Décidez maintenant qui est le **membre A** (crée le dépôt, pousse le kit) et qui est le **membre B** (clone). Les étapes 4 et 5 en dépendent.

Étape 1. Décompresser le kit

5 min, chacun

```
unzip sentiment-app.zip          # Windows : clic droit, Extraire tout
cd sentiment-app
ls
ls -Force | findstr gitignore    # Linux, macOS : ls -a | grep gitignore
```

Contrôle T0 : le kit est complet

Les sept dossiers `app`, `data`, `docs`, `models`, `scripts`, `tests`, `ui`, les fichiers de documentation, et `.gitignore`, qui ne s'affiche pas avec un `ls` simple.

Toutes les commandes du TP partent de ce dossier, celui qui contient `README.md`.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1        # Linux, macOS : source .venv/bin/
    activate
python -m pip install pandas scikit-learn pytest
python -m pytest -q
```

Contrôle T1 : l'état de départ

3 failed, 31 passed, 4 skipped. Les 3 échecs sont les fiches vides (étape 3), les 4 ignorés attendent le dépôt Git (étapes 4 et 6).

Votre invite doit afficher `(.venv)`. L'activation ne vaut que pour la fenêtre courante.

Étape 3. Remplir les trois fiches

50 min, à deux

Lire d'abord `docs/cadrage/EXEMPLE-corrige.md`, puis douze minutes par fiche.

Fiche	Cas
`cas1-pharmacie.md`	détecter les ordonnances incomplètes
`cas2-reglement.md`	répondre aux questions sur le règlement intérieur
`cas3-avis-negatifs.md`	prioriser les avis clients négatifs

Un membre tient le clavier, l'autre parcourt l'arbre de choix et conteste. Rôles inversés à chaque fiche.

Étape 3. Les trois règles que les tests vérifient

1. une seule case cochée par fiche : `- [ ]` devient `- [x]` ;
2. la justification cite au moins deux critères **par leur lettre** : `(a)`, `(b)`, `(c)`, `(d)` ;
3. chaque rubrique obligatoire contient au moins une phrase complète.

```
python -m pytest tests\test_cadrage.py -q
```

Contrôle T2 : les fiches passent

12 passed. Les messages d'échec nomment la fiche et la rubrique fautives.

Écrire « il y a des données étiquetées » sans la lettre `(a)` fait échouer le test.

Sur github.com : New repository, **Private**, aucune case cochée. Settings, Collaborators : membre B en **Write**, enseignant en **Read**.

```
git init
git add -A
git status
```

T3 : le seul vrai piège du TP

La sortie de `git status` ne doit contenir **aucune ligne commençant par** `.venv/`. Si elle en contient, le `.gitignore` manque : arrêtez tout. Un dépôt où `.venv/` entre dans l'historique pèse 300 Mo, et le test final reste rouge même après coup.

Étape 4. Le premier push

```
git commit -m "Semaine 1 : structure du depot et kit TP1"
git branch -M main
git remote add origin https://github.com/<votre-compte>/sentiment-app.git
git push -u origin main
```

Le mot de passe GitHub ne fonctionne pas

Il faut un **jeton d'accès personnel** : GitHub, Settings, Developer settings, Personal access tokens, Tokens (classic), portée `repo`. Copiez-le immédiatement, il ne s'affiche qu'une fois.

Raccourci fourni : `bash scripts/init_depot.sh <url>` fait ces cinq commandes et refuse de pousser si `.venv/` est en attente.

Membre B : acceptez d'abord l'invitation reçue par courriel, sinon le clone échoue avec 403 Forbidden.

```
git clone https://github.com/<votre-compte>/sentiment-app.git
cd sentiment-app
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1    # Windows
bash scripts/setup.sh                                         # Linux, macOS,
     Git Bash
```

Contrôle T4 : le diagnostic

0 erreur(s). L'avertissement « 1 auteur(s) seulement » est normal ici : il disparaît à l'étape 6.

Comptez 5 à 10 minutes : `requirements.txt` installe les paquets de tout le semestre.

```
git pull
# ajouter sa ligne au tableau du binôme dans README.md
git add README.md
git commit -m "README : ajout de <nom> au tableau du binome"
git push
```

Contrôle T5 : les deux membres sont comptés

`git shortlog -sn -all` affiche deux noms distincts, puis `pytest tests/test_git_history.py -q` donne **3 passed**.

Conflit sur `README.md` ? Gardez les deux lignes, supprimez les marqueurs, puis `add`, `commit`, `push`.

Étape 7. Vérification finale

10 min, à deux

```
git pull
python scripts\check_setup.py
python -m pytest -q
```

Contrôle T6 : tout est vert

0 erreur(s), 0 avertissement(s) au diagnostic et **38 passed** (23 d'environnement, 12 de cadrage, 3 d'historique Git).

Poussez une dernière fois, puis vérifiez l'onglet **Actions** : la CI rejoue les mêmes 38 tests sur une machine neuve, en Python 3.11.

Étape 8. Le test de l'enseignant — 5 min

La note se joue sur un clone frais : reproduisez-le avant de partir.

```
cd $env:TEMP
git clone https://github.com/<votre-compte>/sentiment-app.git verif
cd verif
py -3.11 -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scripts\check_setup.py
python -m pytest -q
```

Contrôle T7 : le dépôt tient debout tout seul

0 erreur(s) et **38 passed**, sans aucune intervention manuelle.

Raccourci : `bash scripts/verif_clone.sh <url>`, qui nettoie le dossier temporaire tout seul.

Récapitulatif des huit contrôles

	Commande	Résultat attendu
T0	`ls` et `ls -a`	les 7 dossiers, `.gitignore` présent
T1	`pytest -q`	3 failed, 31 passed, 4 skipped
T2	`pytest tests/test_cadrage.py -q`	12 passed
T3	`git status` avant le commit	aucune ligne `.venv/`
T4	`python scripts/check_setup.py`	0 erreur(s)
T5	`git shortlog -sn -all` puis `pytest tests/test_git_history.py`	deux noms, puis 3 passed
T6	`check_setup.py` et `pytest -q`	0 erreur, 0 avertissement, 38 passed, CI verte
T7	la suite complète sur un clone frais	0 erreur et 38 passed

Les erreurs qui coûtent du temps

Message	Correction
`pytest n'est pas reconnu`	activer le venv, ou écrire `python -m pytest`
`Activate.ps1 cannot be loaded`	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`ModuleNotFoundError: pandas`	venv non activé : `where python` doit contenir `.venv`
`error: 400` sur l'URL	les chevrons `<...>` sont restés dans l'adresse
`Password authentication is not supported`	jeton d'accès personnel, ou `gh auth login`
`403 Forbidden`	invitation de collaborateur non acceptée
`rejected, non-fast-forward`	`git pull`, résoudre le conflit, `git push`
`1 auteur(s) de commits, 2 attendus`	`user.name` distinct sur chaque poste, puis un vrai commit

Authentification (« qui êtes-vous ? ») et autorisation (« avez-vous le droit ? ») : deux erreurs différentes.

Pour aller plus loin, une fois le TP rendu

```
python exemples/train_sentiment.py       # ce que sera la semaine 2
streamlit run exemples/app_streamlit.py  # ce que sera la semaine 5
```

Le premier télécharge 20 000 avis publics (jeu Allociné), entraîne un pipeline TF-IDF et régression logistique, et obtient :

Exactitude	Rappel sur la classe négative	Seuil de la fiche
0,920	0,911	0,90 : accepté

Un classifieur linéaire, quelques secondes, aucun GPU : voilà pourquoi la fiche du cas 3 cochait « machine learning » et non « modèle génératif ».

Avant de quitter la salle

- ☐ Les trois fiches `docs/cadrage/cas*.md` remplies et poussées
- ☐ Le tableau du binôme complété dans `README.md`
- ☐ `requirements.lock` présent dans le dépôt
- ☐ `git shortlog -sn` liste les deux membres
- ☐ `pytest -q` affiche 38 passed
- ☐ L'onglet Actions est vert
- ☐ Le test du clone frais (T7) est passé

Documentation du kit : `MARCHE-A-SUIVRE.md`, `TESTS.md`, `AIDE-GIT.md`, `DEPANNAGE.md`.